

CMPE412 – Project

Distributed Inventory Management with Personalized Verification

Due Date: May 19, 2026

Overview

You will implement a **client-server system** for managing product inventory data. The server hosts multiple inventory files (one per category). Clients connect from a different PC to view product lists or request statistical overviews.

Server Responsibilities

1. **Host at least 4 inventory files**, each containing **≥ 30 product records** in the format:

ProductID, ProductName, Price

Example:

```
P1001, Wireless Mouse, 24.99
P1002, Mechanical Keyboard, 79.50
P1003, USB Cable, 8.75
```

- Categories: *Electronics.txt*, *Clothing.txt*, *Groceries.txt*, *Books.txt*, ...
 - **Each student must generate their own data files** using a script that embeds their student ID as a seed (see *Personalization* section).
2. **On client connection:**
 - Send the list of available file names.
 - Receive client's file choice → send full content of that file.
 - Receive client's "Overview" request → compute and send statistics (see thread pool below).

3. Thread Pool for background processing:

- Read **all inventory files** concurrently.
- For each product, keep **only the highest price** (deduplication).

Examples:

```
Electronics.txt: P1001, Wireless Mouse, 24.99  
Computers.txt: P1008, Wireless Mouse, 22.50  
Accessories.txt: P1081, Wireless Mouse, 27.99 ← keep
```

- Write the merged records into a single file `MergedInventory.txt`.
- From that merged file, compute:
 - **Overall average price**
 - **Highest price** (with product name)
 - **Lowest price** (with product name)
- The thread pool must handle at least 4 threads.

4. GUI (Server Panel)

- IP address display (auto-detected)
- Port input field (default must be based on student ID – see below)
- Start / Stop buttons
- Status label (red/green)

5. Activity log (text area) with “Clear Log” button

Log entries:

```
Server started on port X,  
Client connected from IP,  
Sending file list,  
Client requested file Y,  
Client disconnected,  
Server stopped
```

6. Error handling: file not found, socket errors, invalid requests.

Client Responsibilities

1. GUI (Client Panel)

- **Connection Panel:** server IP field, port field, Connect / Disconnect buttons, status label.
- **File Selection Panel:** dynamic buttons (one per course file received from server) + a **"Get Overview"** button.
- **Data Display Area:**
 - Table to show product records (Product ID, Name, Price) when a file button is clicked.
 - Text area to show overview (average, highest, lowest) when "Get Overview" is clicked.

2. Functionality:

- Connect to server from a **different PC** (or same machine for initial testing, but final demo on two PCs).
 - Receive file list and generate buttons accordingly.
 - On file button click → fetch and display that file's content in the table.
 - On "Get Overview" → request statistics from server and show them in the text area.
-

Personalization & Anti-Plagiarism Measures

Each student's implementation **must** include all of the following to prove individual work:

1. Student-ID-based Port Number

- The server's default port shall be:
$$5000 + (\text{last 4 digits of your student ID}) \% 1000$$

Example: ID 202312345 → last 4 digits 2345 → port $5000 + 345 = 5345$.
- The client must connect using this port (user can override, but default must match the formula).
- **Verification:** During demo, show your student ID and the port displayed in the server GUI

2. Embedded Student ID in Inventory Files

- Use your **full student ID** as a seed to generate the 4 inventory files.
- Each file must contain **exactly one record where the ProductName includes your student ID** (e.g., P9999, StudentID_202312345_Demo, 0.01).
- This record must appear in the merged file after deduplication.
- **Verification:** Instructor will ask you to locate that record in the client's table display.

3. Verification Button on Client GUI

- Add a **"Verify"** button next to the connection panel.
- When clicked, the client sends a special `VERIFY` command to the server.
- The server responds with a **hash** computed as:
$$(\text{sum of digits of your student ID}) * (\text{current active port}) \% 1000$$

(e.g., ID 202312345 → digit sum = 2+0+2+3+1+2+3+4+5 = 22; port 5345 → 22*5345 = 117590 → mod 1000 = 590)
- The client displays this hash in a popup or status bar.

4. Unique Log Entry

- When the server starts, it must write to the activity log:
`"Server started by Student [Your Name] (ID: [Your ID])"`

Technical Requirements

- **Network communication** between two different PCs (Java sockets).
- **GUI** for both server and client (Swing or JavaFX).
- **Thread pool** (`ExecutorService`) for the merging/analysis task.
- **Input/Output streams** (object or text-based) for data transfer.
- **Exception handling** for all network/file errors.
- Clean, documented code (comments explaining the personalization steps).

Deliverables & Demo

1. **Source code** (all `.java` files) + the generated inventory files (`.txt`).
2. **Report** explaining:
 - How your port number is derived from your student ID.
 - How you seeded the inventory files.
 - How the verification hash is computed.
 - A screenshot of the server log showing the personalized startup message.
3. **Live demo** (in lab on May 19 – May 22, 2026):
 - Show server and client running on two different PCs.
 - **STUDENTS MAY PRESENT IN PAIRS.**
 - Demonstrate file list retrieval, file display, and overview.
 - Activate the “Verify” button and show the matching hash.
 - Show the special record containing your student ID.

No demo = no grade. The personalization features will be cross-checked with your submitted student ID.

Grading Rubric

Component	Points
Server GUI (start/stop, log, port based on ID)	15
Client GUI (connection, dynamic buttons, table, overview area)	15
Correct file transfer & thread-pool based merging/deduplication	25
Overview statistics (avg, max, min)	10
Personalization (port formula, seeded files, verification button, log entry)	20
Successful remote communication (two PCs)	10
Code quality, comments, error handling	5
Total	100